

Documento Técnico de Arquitectura

Generative Code Checker - Sistema Inteligente de Revisión de Código

1. Visión General del Sistema

El **Generative Code Checker** es una solución de software diseñada para analizar, diagnosticar y recomendar mejoras sobre fragmentos de código fuente enviados por estudiantes. El sistema centraliza el flujo de revisión integrando modelos de Inteligencia Artificial (LLM) mediante un pipeline estructurado, garantizando la trazabilidad inmutable de las revisiones, auditoría de métricas y la separación clara de responsabilidades en la arquitectura.

2. Arquitectura y Componentes

La solución está estructurada en un modelo cliente-servidor tradicional, separando de manera estricta la interfaz de usuario, la capa de lógica de negocio/middleware y la persistencia de datos. Esta arquitectura asegura escalabilidad, facilidad para pruebas automatizadas y un control transaccional riguroso.

2.1. Frontend (Capa de Presentación)

- **Stack:** React y Tailwind CSS.
- **Responsabilidad:** Renderizado de la interfaz de usuario, captura de inputs del estudiante (lenguaje, nivel, objetivo, código), visualización de diagnósticos a través de un *diff viewer*, y despliegue del Dashboard analítico.
- **Interacción:** Se comunica exclusivamente a través de endpoints REST expuestos por el Backend, manteniendo la independencia de la lógica central.

2.2. Backend y Middleware (Capa Lógica Core)

- **Stack:** Python (Flask/Django).
- **Responsabilidad:** Orquestación central de solicitudes, validación perimetral de inputs, y manejo de la seguridad (variables de entorno, exclusión de Git).
- **Manejo de Errores y Parseo Defensivo:** Uno de los pilares del backend es la resolución programática de problemas. Al depender de las respuestas de la IA, el middleware

incorpora rutinas de parseo defensivo. Si el modelo devuelve un JSON malformado, truncado o con alucinaciones, el parser interviene para intentar reconstruirlo o, como contingencia, falla de manera controlada devolviendo un error amigable al cliente sin romper la aplicación. Este enfoque garantiza alta disponibilidad y consistencia de datos.

2.3. Base de Datos (Capa de Persistencia)

- **Stack:** Supabase (PostgreSQL).
- **Responsabilidad:** Persistencia transaccional de auditorías, historial de prompts, diagnósticos y vistas para métricas de agregación SQL del Dashboard.

3. Decisiones Técnicas y Justificación del Stack

La selección de tecnologías responde directamente a la necesidad de alto rendimiento, despliegue rápido y una arquitectura orientada al backend robusto.

Componente	Tecnología Elegida	Justificación Técnica
Backend Framework	Python (Flask/Django)	El ecosistema de Python permite un enfoque profundo en el desarrollo backend, permitiendo construir módulos de validación perimetral y conectores de IA robustos centrados en resolver la lógica y estructura defensiva del sistema.
Base de Datos	PostgreSQL (Supabase)	La rúbrica exige trazabilidad inmutable y consultas analíticas estructuradas. Se determinó que un modelo estrictamente relacional como PostgreSQL es mandatorio, descartando

Componente	Tecnología Elegida	Justificación Técnica
		completamente opciones NoSQL como MongoDB, que carecen de la rigidez de esquemas y transacciones relacionales necesarias para este control de auditoría.
Frontend UI	React + Tailwind CSS	Genera un visor comparativo (diff viewer) ágil y consume de manera eficiente la API del backend sin sobrecargar la capa de presentación.

4. Integración con Inteligencia Artificial (IA Generativa)

La inteligencia artificial es el motor analítico del sistema. No funciona de forma decorativa, sino que recibe el código y contexto para retornar diagnósticos procesables.

4.1. Flujo de Servicio LLM

1. **Input Processing:** El backend intercepta los datos desde React, normaliza el texto y estructura las variables (lenguaje, nivel, objetivo).
2. **Prompt Builder:** Inyecta las variables en un *System Prompt* versionado. Incorpora restricciones pedagógicas obligatorias, ordenando explícitamente a la IA que no asuma que el código es ejecutable ni afirme su funcionamiento sin pruebas de sandbox.
3. **Structured Output (JSON Schema):** Se obliga al SDK de la IA a ceñirse al `response_schema.json v1.0`. La salida se estructura rígidamente en: *summary*, *findings*, *explanation*, *suggested_code*, *tests*, *warnings*.
4. **Validación y Reparación:** El Middleware inspecciona la estructura devuelta antes de permitir su acceso a Supabase o al Frontend.

5. Configuración, Servicios y Seguridad

El manejo de entorno es un requerimiento crítico para el sistema.

5.1. Variables de Entorno

La configuración se gestiona por completo mediante un archivo `.env` aislado del repositorio mediante el `.gitignore`. Este incluye:

```
# Configuración del LLM
LLM_API_KEY=your_secure_api_key
LLM_MODEL_VERSION=gpt-4o-or-gemini

# Configuración de Supabase
SUPABASE_URL=your_supabase_project_url
SUPABASE_SERVICE_ROLE_KEY=your_database_key
```

5.2. Servicios y Trazabilidad

Cada transacción inicia con la llamada de la API del LLM e inmediatamente se registra tanto el prompt exacto enviado, como la respuesta cruda recibida en una tabla relacional de *logs* en Supabase. Las vistas SQL analíticas pre-calculan las métricas para alimentar el Dashboard React en tiempo real, garantizando rendimiento y aislamiento de responsabilidades entre servicios.